Project: Serverless & Microservice Patterns

Due: 02/28/2026 11:59 PM
Extended Deadline: 03/01/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

knative
- Task Overview
- Task

grpc
- Task Overview
- Task

dapr
- Task Overview
- Task

Wrap Up
- Conclusion
- End of project survey

knative (Task)

Introduction

Information

If you haven't read the section on `Contexts and Pre-requisites`, we encourage you to do so before commencing this project.

README File

Please add an author section to your README file. Put your real name in there, soon we'll be reviewing your answers.txt files and we want to know whos's answering the questions.

In traditional Kubernetes projects, you had to manage multiple resources yourself—Deployments for pods, Services and LoadBalancers for access, Ingress for routing, and HPAs for scaling.

This meant a lot of YAML and operational glue just to run and scale an application correctly.
Knative is a Kubernetes-based platform that abstracts this heavy lifting by managing deployment, scaling, and traffic routing for you. Instead of wiring everything together, you define your application once and Knative handles running it on demand, scaling it automatically, and routing traffic safely.

Knative, as a FaaS platform (Functions as a Service), excels for background jobs that are CPU-intensive. They play well with systems like Kubernetes Event-Driven Autoscaling and are the "correct" way to architect a system like this. Most FaaS platforms can simplify development speed tremendously, at the cost of some vendor lock-in, as we will be depending on the vendor's runtime. Luckily, this runtime is open source, and can be hosted on AKS.

Once you install Knative on your cluster. You may create any number of Knative functions (also called services). These services handle the deployment and scaling of your docker container.

We recommend you go through the official documentation on Knative for more information.

Project Setup Instructions

The Serverless & Microservices Patterns is a unique project and requires a special setup.
You'll only use it once, hence, you'll be creating a separate branch on which you'll work.
You will not work on the main branch for this project. Let's go through these instructions carefully:

1. Create a new branch, `serverless-and-microservice-patterns`.
2. Switch to this new branch with the `git switch` command
3. Confirm you're in the `serverless-and-microservice-patterns` using the **git branch** command.
4. In the project's root directory, create a new directory, `project3`
5. In the `project3` directory, create a sub-directory, `task1`
6. You'll work in the **project3/task1** directory for this task only.

Assumptions

This task assumes the following:

1. You have already provisioned an Azure Container Registry (ACR) and a Kubernetes Cluster
2. Make sure your cluster has `3 Nodes` in the Pool. If you set it to anything less than 3 for previous projects, now is a good time to switch to 3.
3. You have built and pushed your container image to the ACR

Goal

The goal of this task is to deploy and scale your LLM service using Knative. A high level of the task is as follows:

1. Install the `Knative Operator`. This installs and manages Knative for you.
2. Configure a Knative serving resource.
3. Install Kourier
4. Configure a Knative serving object
5. Load test your microservice

Sub-task 1 - Install Knative Operator

You need a Knative operator to install, configure and manage Knative for you. You will install the Knative operator by running the command below:

```
kubectl apply -f https://github.com/knative/oper
```

Sub-task 2 - Install Knative Serving

Knative Serving deploys and manages serverless applications on Kubernetes, enabling automatic scaling based on traffic.

Create a file named `knative-serving.yaml` in the `task1` folder. You will put all knative-related files here.
You will put all YAML manifests for this task in the `project3/task1` directory.

Copy the following code section into a yaml file named `knative-serving.yaml` file -

```
# YAML file -
apiVersion: v1
kind: Namespace
metadata:
  name: knative-serving
---
apiVersion: operator.knative.dev/v1beta1
kind: KnativeServing
metadata:
  name: knative-serving
  namespace: knative-serving
spec:
  ingress:
    kourier:
      enabled: true
  config:
    network:
      ingress-class: "kourier.ingress.networking
```

Run the following command to deploy the object:

```
kubectl apply -f knative-serving.yaml
```

Verify Installation

```
kubectl get deployment -n knative-serving
```

If Knative Serving has been successfully deployed, all deployments of the Knative Serving will show READY status. Here is a sample output:

```
NAME                     READY   UP-TO-DATE   AV
3scale-kourier-gateway   1/1     1            1
activator                1/1     1            1
autoscaler               1/1     1            1
autoscaler-hpa           1/1     1            1
controller               1/1     1            1
```

```
net-kourier-controller   1/1     1            1
webhook                  1/1     1            1
```

Information

Note that the `3scale-kourier-gateway` resource takes time to spin up. Roughly 1-2 minutes. If it's taking too long to provision, make sure that your cluster has 3 nodes running.

Sub-task 3 - Networking Using Kourier

For the serverless application to be reachable from the outside, the requests are handled through a gateway. The requests are routed to the gateway and the gateway routes the request to the application we deployed. In this project, we use Kourier for our networking layer. If you notice the previous contents of the yaml file, we have enabled kourier for our ingress.

Configure DNS

To configure `Kourier`, run the following command:

```
kubectl apply -f https://github.com/knative/serv
```

Sub-task 4 - Develop the Knative serving object

This knative serving object is the serverless function. Copy the following contents into a `serverless.yaml` file and deploy it. Make sure to set the image to the image you deployed to your ACR.

Remember that, to deploy, you use the command: `kubectl apply -f <file-name>`

```
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: llm-service
  namespace: default
spec:
  template:
    spec:
      containers:
      - image: llmacrfs26.azurecr.io/llmappprod:
        ports:
          - containerPort: 8000
        readinessProbe:
          httpGet:
            path: /healthcheck
```

```
            initialDelaySeconds: 15
            periodSeconds: 10
            timeoutSeconds: 5
```

Once deployed, you should be able to see a ksvc object by running the following command - `kubectl get ksvc llm-service`. You should see an output like the example below -

```
NAME          URL
llm-service   http://llm-service.default.57.152.
```

The URL obtained from this GET command will be the URL which you can use to reach out to your endpoint.

Verification -

HTTPS:

```
curl http://helloworld-python.default.1.2.3.4.ss
```

```
curl http://helloworld-python.default.1.2.3.4.ss
```

Ensure that you update the above links with the URL displayed after running the `kubectl get ksvc llm-service` command.

Sub-task 5 - Load Testing

The true power of Knative becomes clear under load. Sending a few `curl` requests to your service only verifies that it works — it does not demonstrate what Knative is designed to do: automatically scale up under demand and scale back down when traffic stops.

To properly observe Knative's autoscaling behavior, we'll use **Locust**, a Python-based load testing tool. Locust allows you to simulate many concurrent users sending requests to your service over a sustained period of time. This more accurately reflects real-world traffic patterns, where multiple users interact with an application simultaneously.

By generating controlled load with Locust, you'll be able to:

- Observe Knative scale your pods up as traffic increases
- Measure response times under concurrency
- Watch the service scale back down when traffic drops
- Understand how request volume impacts performance

You will follow the instructions below to create your first locust file and stress test your application.

```
from locust import HttpUser, task
from wonderwords import RandomWord


class HelloWorldUser(HttpUser):
    @task
    def hello_world(self):
        w = RandomWord()
        self.client.get(f"/api?message={w.word()


"""
USAGE

This is a simple load testing tool using Locust.

1. Create a directory named `loadtest` in your p
      mkdir loadtest
      cd loadtest

2. Create a file named `locustfile.py` and paste

3. Create a `requirements.txt` file with the fol
      locust
      wonderwords

4. Create a virtual environment:
      python3 -m venv .venv

5. Activate the virtual environment:
      source .venv/bin/activate

6. Install dependencies:
      pip install -r requirements.txt

7. Run the load test (replace <URL> with your se
      locust -f locustfile.py -H <URL> -u 5 -r (

Example:
      locust -f locustfile.py -H http://localhos
"""
```

You should see an output similar to the following after executing the run command:

```
(.venv) azureuser@qa-vm:~/cloud-native-handout-h
 --run-time 3m
[2026-02-16 22:50:27,401] qa-vm/INFO/locust.main
```

```
[2026-02-16 22:50:27,402] qa-vm/INFO/locust.mair
```

Make sure you're connected to your VM using VS Code. VS Code does the port forwarding for you so you can run the test from an interactive browser window locally.

Press the ENTER key to proceed.

You'll be presented with a menu, click on **START** to commence the load test.

Activity 1

While your load test is running, make the following observations:

1. How many pods were running before executing the load test?

```
kubectl get pod
```

1. How many pods were span during the load test?
2. Upon running this, new pods are automatically created based on the traffic. However, the scaling may require further fine-tuning to suit our use case. For instance, we may want to set the concurrency to 1 per pod. To achieve this, we can add the following annotation to the service (`serverless.yaml` file). Don't forget to run `kubectl apply -f serverless.yaml` after the update.

```
spec:
    template:
        metadata:
            annotations:
                autoscaling.knative.dev/target:
```

1. Run the load test again and report your findings in the `answers/unit2answers.txt` file.

Reflection Questions

1. Did you realize after the load test, when you run `kubectly get pods`, you see no pods in the cluster? Explain why this is so and what benefits it brings.
2. The locust dashboard has a number of charts. Can you interprete what each chart represents? Feel free to check online or use Generative AI tools.
3. Can you identify three real-world workloads that will benefit from the capabilities of knative?
4. What sort of workloads are not ideal for knative?

5. Add your responses to the above questions in the `answers/unit2answers.txt` file.

[Optional] Activity 2

If you were paying close attention, you likely noticed that you used your own VM to generate load during the load testing activity. In real-world production environments, teams typically rely on dedicated, high-performance machines to execute load tests at scale.

Fortunately, Azure makes this easy. Provision a high-performance VM on Azure and run a Locust test from that server for approximately 30 minutes, bump the number of concurrent users to around 50. Then observe how effectively your microservice scales under sustained load.

Don't forget to delete this VM after your tests run. Leaving an expensive resource running for too long will hurt your Azure credits.

Tree Structure for this Task

```
task1
├── knative-serving.yaml
└── serverless.yaml
```

Validate

Validate that you have built this all as expected by curling into your service and making sure you can access the model.

Once this is done, navigate to your project root directory and you should be able to submit by using:

```
./submitter task1
```

Terminate your Resources

Once you are done with this task, ensure that you have terminated all resources used for this project. The AKS Cluster has 3 nodes which is costly. Please delete these resources once done.

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform